

In[13]:=

```
(* Pull real 10m elevation data for Durham, NC *)
demRaw = GeoElevationData[
  Entity["City", {"Durham", "NorthCarolina", "UnitedStates"}],
  GeoRange → Quantity[10, "Kilometers"],
  GeoProjection → Automatic
];

(* Strip units – GeoElevationData returns a QuantityArray *)
dem = QuantityMagnitude[demRaw];

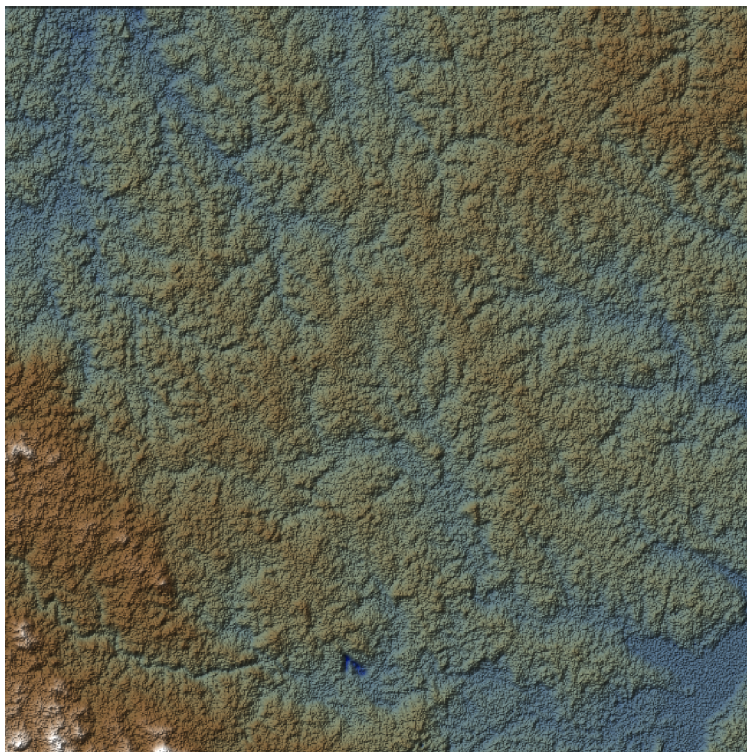
(* Simulate heavy artifacts: random pits scattered across the DEM *)
SeedRandom[42];
{nr, nc} = Dimensions[dem];
rawDEM = dem;
(* Add 300 random deep pits – drop elevation by 30-80 units *)
pitPositions = RandomInteger[{1, Min[nr, nc]}, {300, 2}];
Do[
  rawDEM[[pt[[1]], pt[[2]]]] -= RandomReal[{30, 80}],
  {pt, pitPositions}
];
(* Also add moderate noise everywhere *)
rawDEM = rawDEM + RandomReal[{-8, 8}, {nr, nc}];

(* Conditioned: heavy smoothing to remove pits and depressions *)
condDEM = MedianFilter[GaussianFilter[dem, 3], 2];

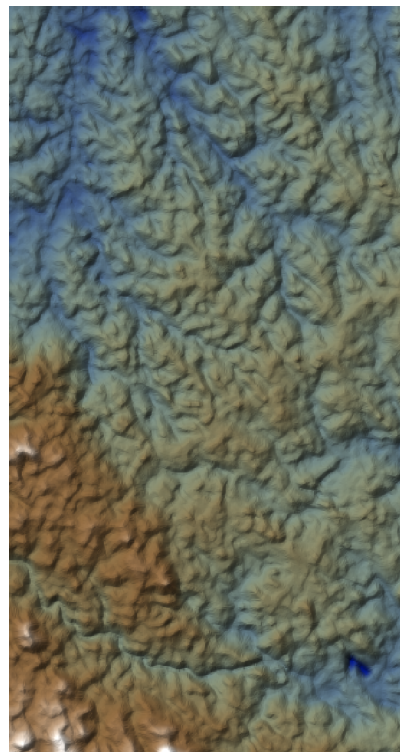
GraphicsRow[{
  ReliefPlot[rawDEM,
    ColorFunction → "DarkTerrain",
    PlotLabel → Style["Raw DEM (with artifacts)", 12, Bold],
    Frame → False, ImageSize → 400],
  ReliefPlot[condDEM,
    ColorFunction → "DarkTerrain",
    PlotLabel → Style["Conditioned DEM", 12, Bold],
    Frame → False, ImageSize → 400]
}, Spacings → 20, ImageSize → 850]
```

Out[22]=

**Raw DEM (with artifacts)**



**Conditione**



```

In[23]:= (* D8 flow direction field visualization *)
gridN = 30;
SeedRandom[17];

(* Create a smooth terrain with a valley *)
terrain = Table[
  3 Exp[-((x - 15)^2 + (y - 15)^2)/80] +
  2 Exp[-((x - 8)^2 + (y - 22)^2)/50] -
  1.5 Exp[-((x - 15)^2)/20] +
  RandomReal[{-0.05, 0.05}],
  {y, 1, gridN}, {x, 1, gridN}
];

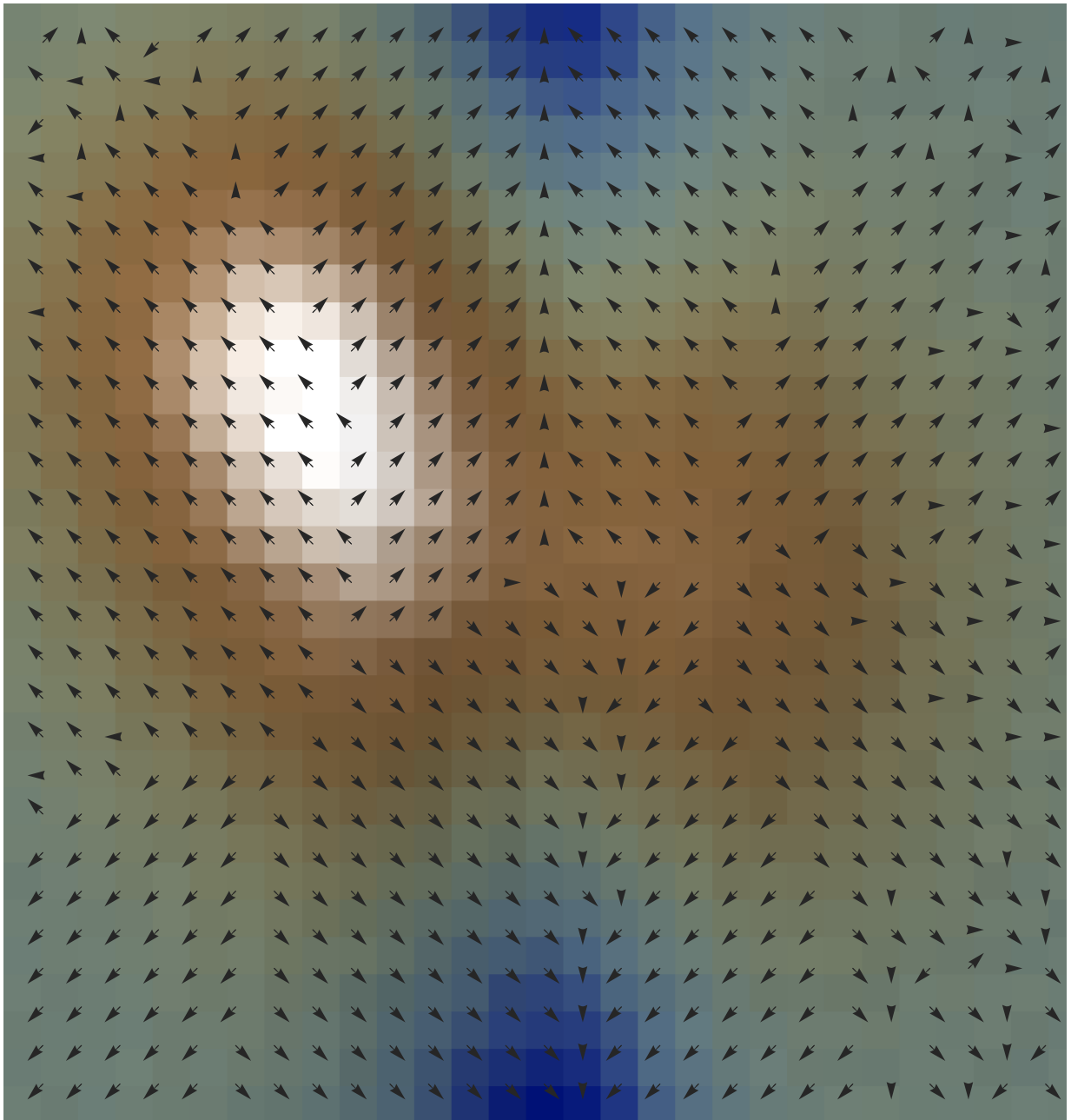
(* Compute D8 directions *)
offsets = {{-1, -1}, {-1, 0}, {-1, 1}, {0, -1},
           {0, 1}, {1, -1}, {1, 0}, {1, 1}};

flowArrows = Flatten@Table[
  Module[{slopes, best, dy, dx},
    slopes = Table[
      If[1 ≤ y + o[[1]] ≤ gridN && 1 ≤ x + o[[2]] ≤ gridN,
        terrain[[y, x]] - terrain[[y + o[[1]], x + o[[2]]],
        -Infinity
      ], {o, offsets}];
    best = FirstPosition[slopes, Max[slopes]][[1]];
    {dy, dx} = offsets[[best]];
    If[Max[slopes] > 0,
      {Arrowheads[0.015],
       Arrow[{x, gridN + 1 - y}, {x + 0.4 dx, gridN + 1 - y - 0.4 dy}]}],
    Nothing
  ],
  {y, 2, gridN - 1}, {x, 2, gridN - 1}
];

Show[
  ReliefPlot[terrain, ColorFunction → "DarkTerrain",
    DataRange → {{1, gridN}, {1, gridN}}, Frame → False],
  Graphics[{GrayLevel[0.15], flowArrows}],
  PlotLabel → Style["D8 Flow Direction Field", 14, Bold],
  ImageSize → 700
]

```

Out[28]=

**D8 Flow Direction Field**



```

In[29]:= (* 3D flow accumulation surface from real elevation data *)
demRaw2 = GeoElevationData[
  Entity["City", {"Durham", "NorthCarolina", "UnitedStates"}],
  GeoRange → Quantity[10, "Kilometers"]
];

(* Strip units *)
dem2 = QuantityMagnitude[demRaw2];

(* Approximate flow accumulation via iterative downhill propagation *)
{ny, nx} = Dimensions[dem2];
accum = ConstantArray[1.0, {ny, nx}];
offsets = {{-1,-1},{-1,0},{-1,1},{0,-1},{0,1},{1,-1},{1,0},{1,1}};

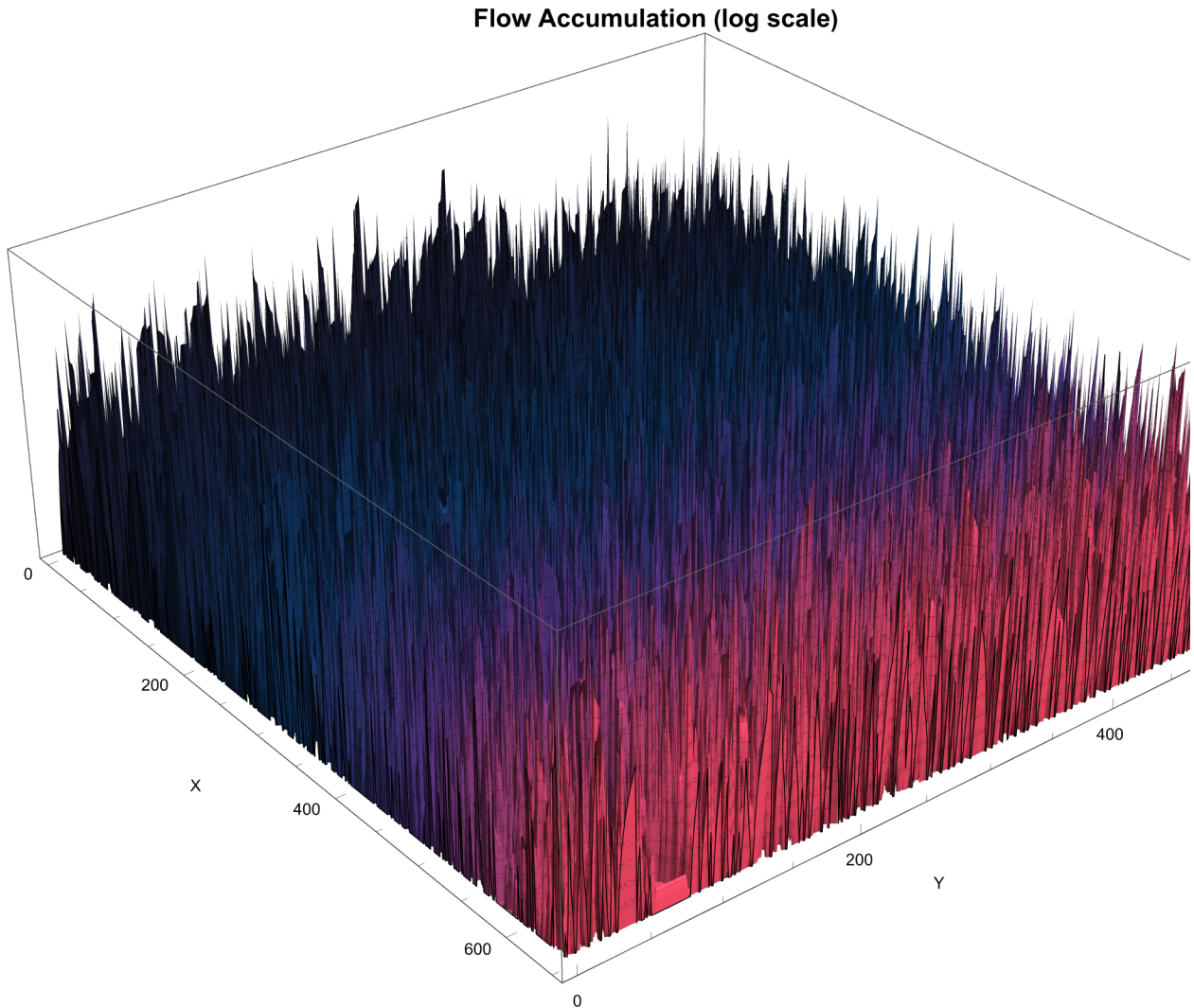
cells = Flatten[Table[{y, x}, {y, ny}, {x, nx}], 1];
sorted = SortBy[cells, -dem2[[#1], #[2]] &];

Do[
  Module[{y = cell[[1]], x = cell[[2]], slopes, best, dy, dx},
    slopes = Table[
      If[1 ≤ y + o[[1]] ≤ ny && 1 ≤ x + o[[2]] ≤ nx,
        dem2[[y, x]] - dem2[[y + o[[1]], x + o[[2]]],
        -Infinity
      ], {o, offsets}];
    If[Max[slopes] > 0,
      best = FirstPosition[slopes, Max[slopes]][[1]];
      {dy, dx} = offsets[[best]];
      accum[[y + dy, x + dx]] += accum[[y, x]];
    ];
  ],
  {cell, sorted}
];

ListPlot3D[Log10[accum + 1],
  PlotRange → All,
  ColorFunction → (Blend[{RGBColor["#1a1a2e"], RGBColor["#16213e"],
    RGBColor["#0f3460"], RGBColor["#533483"], RGBColor["#e94560"]}], #] &),
  ColorFunctionScaling → True,
  MeshFunctions → {#3 &}, MeshStyle → Opacity[0.1],
  BoxRatios → {1, 1, 0.4},
  PlotLabel → Style["Flow Accumulation (log scale)", 14, Bold],
  AxesLabel → {"X", "Y", "log10(Accum)"},
  ViewPoint → {2.5, -2, 1.8},
  Lighting → "Neutral",
  ImageSize → 800
]

```

Out[37]=



```
In[38]:= (* Stream network with Strahler ordering as a tree graph *)
SeedRandom[7];

buildTree[depth_] := If[depth == 0,
  <|"order" → 1, "children" → {}|>,
  Module[{left = buildTree[depth - 1], right = buildTree[depth - 1], ord},
    ord = If[left["order"] == right["order"],
      left["order"] + 1, Max[left["order"], right["order"]]];
    <|"order" → ord, "children" → {left, right}|>
  ]
];

tree = buildTree[5];

edgeList = {};
vertexOrders = <||>;
```

```

posMap = <|>;
idx = 0;

addEdges[node_, parent_, px_, py_, dx_, level_] := Module[
  {myIdx = ++idx, cx = px, cy = py},
  posMap[myIdx] = {cx, cy};
  vertexOrders[myIdx] = node["order"];
  If[parent > 0, AppendTo[edgeList, parent → myIdx]];
  If[node["children"] != {},
    addEdges[node["children"][[1]], myIdx, cx - dx, cy - 1, dx/2, level + 1];
    addEdges[node["children"][[2]], myIdx, cx + dx, cy - 1, dx/2, level + 1];
  ];
];

addEdges[tree, 0, 0, 0, 8, 0];

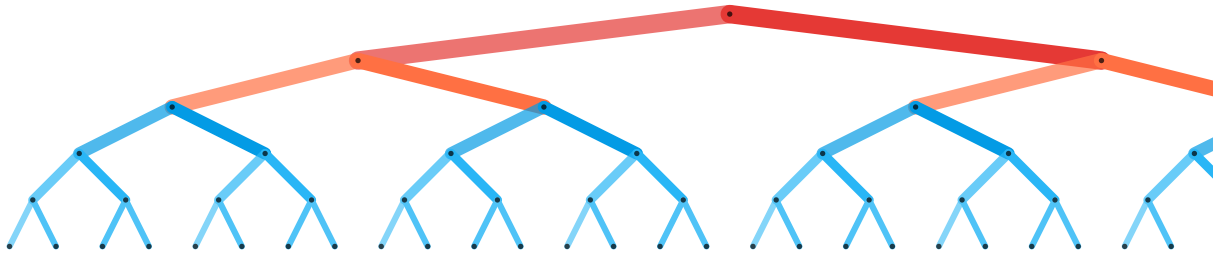
orderColors = <|1 → RGBColor["#4FC3F7"], 2 → RGBColor["#29B6F6"],
  3 → RGBColor["#039BE5"], 4 → RGBColor["#FF7043"],
  5 → RGBColor["#E53935"], 6 → RGBColor["#B71C1C"]>;

Graph[edgeList,
  VertexCoordinates → Normal[posMap],
  EdgeStyle → Directive[Thick],
  EdgeShapeFunction → (
    Module[{ord},
      ord = vertexOrders[#2[[2]]];
      {Thickness[0.002 + 0.002 ord],
        Lookup[orderColors, ord, Gray],
        Line[#1]}
    ] &),
  VertexSize → 0,
  PlotLabel → Style["Stream Network – Strahler Ordering", 14, Bold],
  Epilog → {
    Inset[SwatchLegend[
      Values[orderColors][[1 ;; 5]],
      Style[#, 11] & /@ {"Order 1", "Order 2", "Order 3", "Order 4", "Order 5"},
      LegendLayout → "Row"], Scaled[{0.5, -0.08}]]
  },
  ImageSize → 800, PlotRangePadding → 1
]

```

Out[48]=

## Stream Network — Strahler Ordering



In[59]:=

```
(* Candidate placement along a meandering stream *)
SeedRandom[12];

(* Main river – wide horizontal meanders *)
nPts = 300;
t = Subdivide[0, 10, nPts];
river = Table[{
  s * 3,
  4 Sin[0.8 s] + 2 Sin[1.7 s] + 0.8 Cos[2.5 s]
}, {s, t}];

spacing = 12;
candidates = river[[1 ;; ;; spacing]];

(* Tributary branching off at ~40% along the main river *)
branchStart = river[[120]];
tributary = Table[
  branchStart + {-2.5 s + 1.5 Sin[1.5 s], -s * 3 - 1.2 Cos[2 s]},
  {s, Subdivide[0, 2.5, 80]}
];
tribCandidates = tributary[[1 ;; ;; spacing]];

(* Second smaller tributary *)
branch2 = river[[200]];
trib2 = Table[
  branch2 + {1.5 s + Sin[2 s], s * 2.5 + 0.8 Sin[3 s]},
  {s, Subdivide[0, 2, 50]}
];
trib2Candidates = trib2[[1 ;; ;; spacing]];

Graphics[{
  (* Rivers *)
  {RGBColor["#1565C0"], Thickness[0.004], Line[river]},
  {RGBColor["#42A5F5"], Thickness[0.0025], Line[tributary]},
  {RGBColor["#42A5F5"], Thickness[0.002], Line[trib2]},
}
```

```

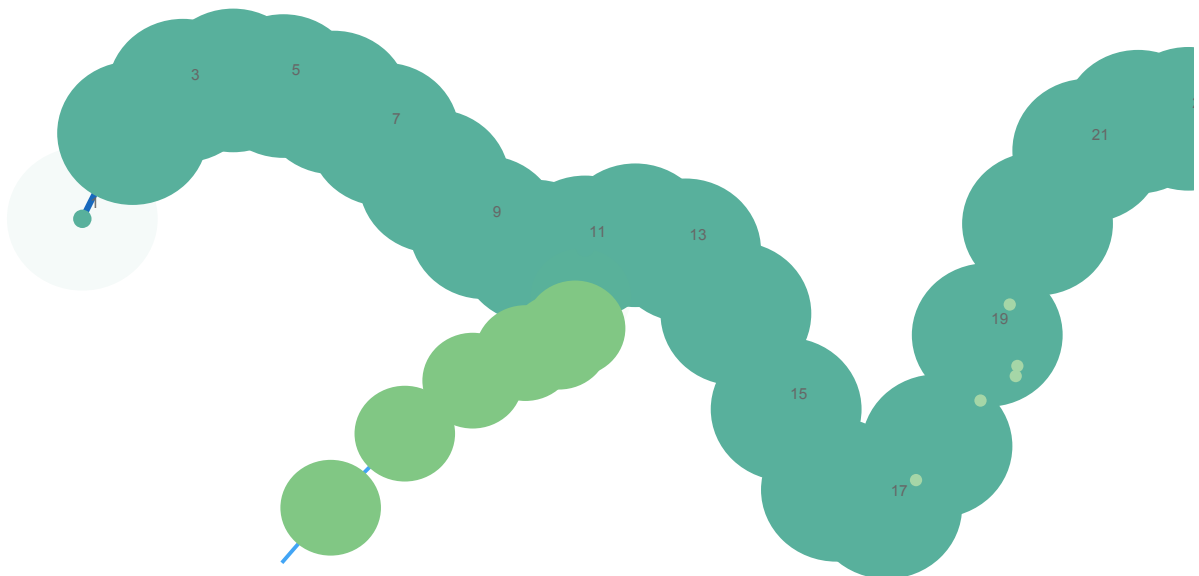
(* Catchment halos *)
{RGBColor["#58B09C"], Opacity[0.06], Disk[#, 1.8] & /@ candidates},
{RGBColor["#81C784"], Opacity[0.05], Disk[#, 1.2] & /@ tribCandidates},

(* Candidate dots *)
{RGBColor["#58B09C"], PointSize[0.012], Point /@ candidates},
{RGBColor["#81C784"], PointSize[0.009], Point /@ tribCandidates},
{RGBColor["#A5D6A7"], PointSize[0.008], Point /@ trib2Candidates},

(* Labels – only label every other candidate to reduce clutter *)
{GrayLevel[0.4],
 MapIndexed[
   If[Mod[#2[[1]], 2] == 1,
     Text[Style[#2[[1]], 8], #1 + {0.3, 0.4}],
     Nothing] &,
   candidates]}
},
PlotLabel → Style["Candidate Net Sites – 200m Spacing", 14, Bold],
Background → White,
PlotRange → All,
PlotRangePadding → 1.5,
ImageSize → 800,
AspectRatio → 0.45
]

```

**Candidate Net Sites — 200m Spacing**



```

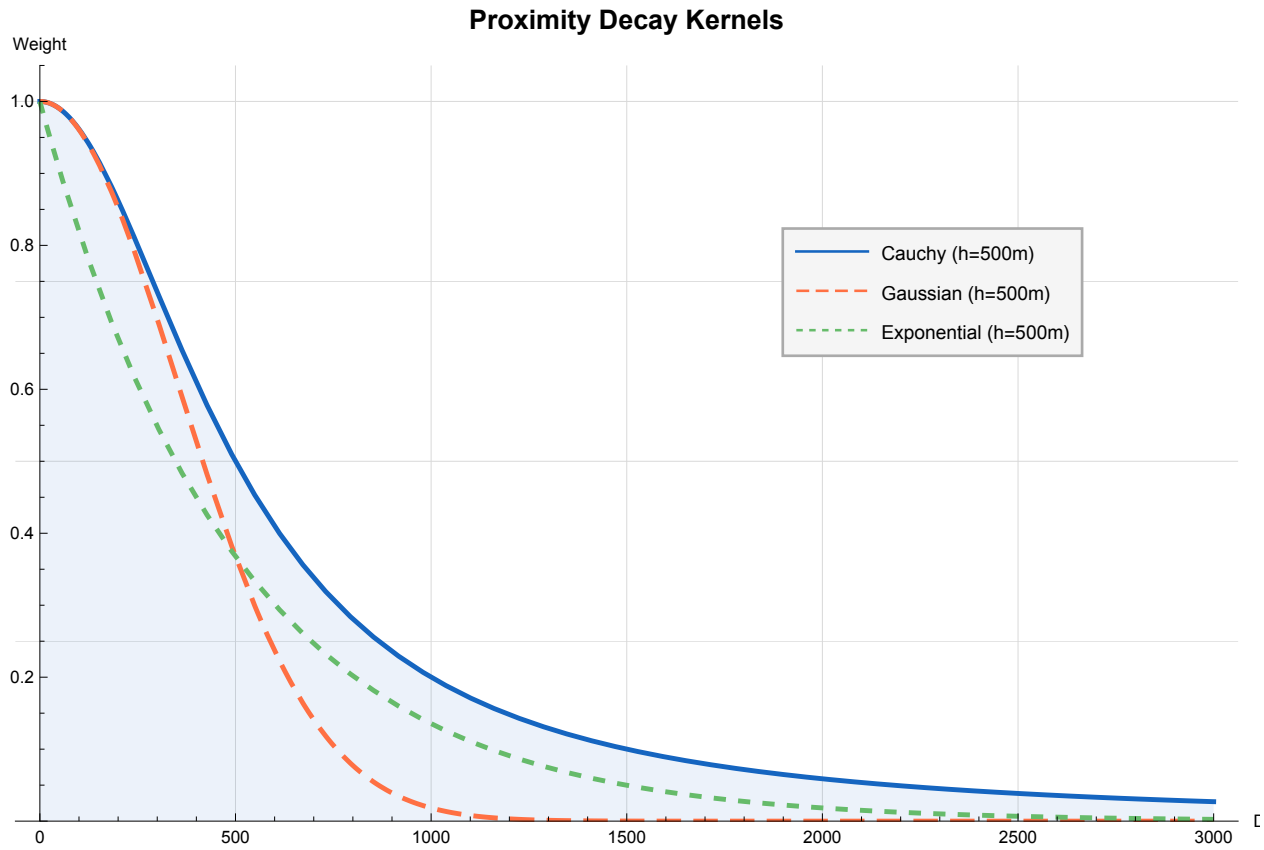
In[72]:= (* Cauchy vs Gaussian vs Exponential decay kernels *)
h = 500;

cauchy[d_] := 1 / (1 + (d/h)^2);
gaussian[d_] := Exp[-(d/h)^2];
exponential[d_] := Exp[-d/h];

Plot[{cauchy[d], gaussian[d], exponential[d]},
  {d, 0, 3000},
  PlotStyle → {
    {RGBColor["#1565C0"], Thickness[0.004]},
    {RGBColor["#FF7043"], Thickness[0.004], Dashing[{0.02, 0.01}]},
    {RGBColor["#66BB6A"], Thickness[0.004], Dashing[{0.01, 0.01]}}
  },
  PlotLegends → Placed[
    LineLegend[{"Cauchy (h=500m)", "Gaussian (h=500m)", "Exponential (h=500m)"},
      LegendFunction → "Panel", LegendLayout → "Column"],
    {0.75, 0.7}
  ],
  AxesLabel → {"Distance (m)", "Weight"},
  PlotLabel → Style["Proximity Decay Kernels", 14, Bold],
  PlotRange → {0, 1.05},
  GridLines → {{500, 1000, 1500, 2000, 2500}, {0.25, 0.5, 0.75, 1.0}},
  GridLinesStyle → Directive[GrayLevel[0.85]],
  Filling → {1 → {Axis, Directive[RGBColor["#1565C0"], Opacity[0.08]]}},
  ImageSize → 700
]

```

Out[76]=





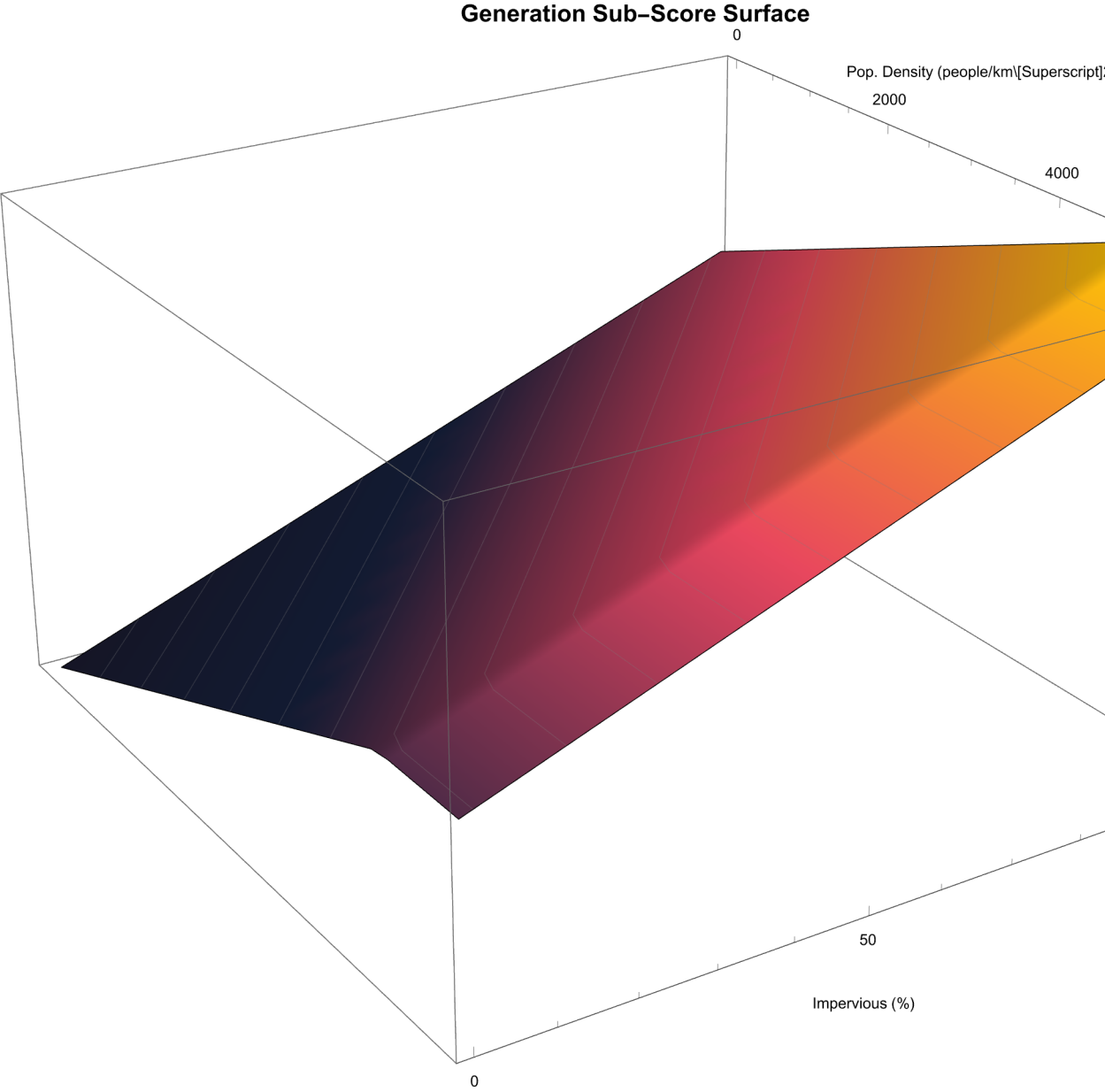
```

In[77]:= (* Generation sub-score surface *)
generationScore[popDens_, impervPct_] := Module[
  {popNorm, impNorm},
  popNorm = Min[popDens / 5000, 1];
  impNorm = impervPct / 100;
  0.18 popNorm + 0.20 impNorm +
    0.62 (0.4 popNorm + 0.6 impNorm)
];

Plot3D[100 generationScore[pop, imp],
  {pop, 0, 6000}, {imp, 0, 100},
  ColorFunction → (Blend[{
    RGBColor["#1a1a2e"], RGBColor["#16213e"],
    RGBColor["#e94560"], RGBColor["#ffcc00"]}], #3] &),
  ColorFunctionScaling → True,
  MeshFunctions → {#3 &},
  MeshStyle → Directive[GrayLevel[0.5], Opacity[0.3]],
  AxesLabel → {"Pop. Density (people/km\[Superscript]2)", "Impervious (%)", "Score"},
  PlotLabel → Style["Generation Sub-Score Surface", 14, Bold],
  BoxRatios → {1, 1, 0.6},
  ViewPoint → {2.8, -1.8, 1.5},
  Lighting → "Neutral",
  PlotRange → {0, 100},
  ImageSize → 800
]

```

Out[78]=



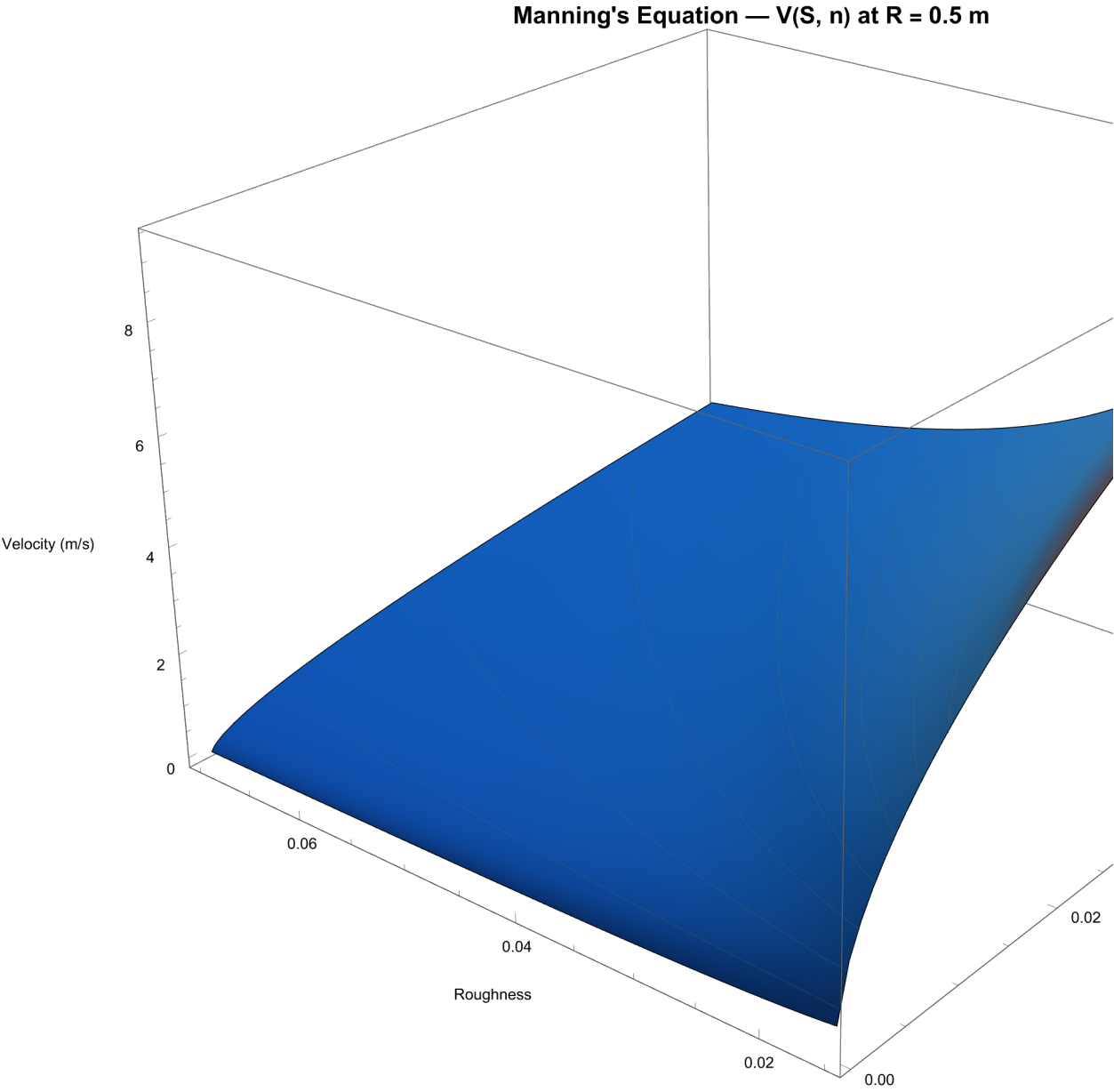
```

In[79]:= (* Manning's velocity as function of slope and roughness *)
manningV[slope_, rough_] := (1/rough) * 0.5^(2/3) * Sqrt[slope];

Plot3D[manningV[slope, rough],
  {slope, 0.0001, 0.05}, {rough, 0.015, 0.070},
  ColorFunction -> (Blend[{
    RGBColor["#0D47A1"], RGBColor["#1565C0"],
    RGBColor["#42A5F5"], RGBColor["#FF7043"],
    RGBColor["#E53935"]}, #3] &),
  ColorFunctionScaling -> True,
  MeshFunctions -> {#3 &},
  MeshStyle -> Directive[GrayLevel[0.4], Opacity[0.3]],
  AxesLabel -> {"Slope", "Roughness", "Velocity (m/s)"},
  PlotLabel -> Style["Manning's Equation - V(S, n) at R = 0.5 m", 14, Bold],
  BoxRatios -> {1, 1, 0.7},
  ViewPoint -> {-2.5, -2, 1.5},
  Lighting -> "Neutral",
  PlotPoints -> 50,
  ImageSize -> 800
]

```

Out[80]=



```

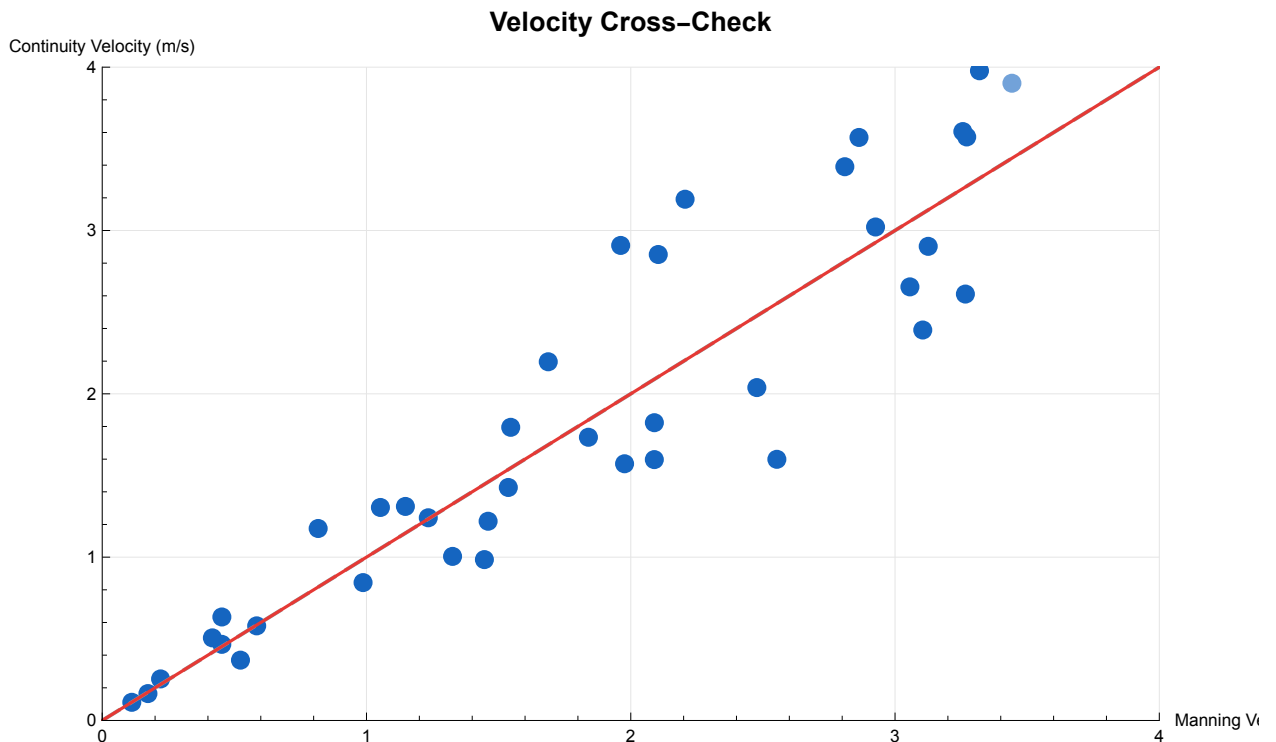
In[81]:= (* Manning vs continuity velocity scatter *)
SeedRandom[99];
nPts = 40;

manningVals = RandomReal[{0.1, 3.5}, nPts];
continuityVals = manningVals * RandomReal[{0.6, 1.5}, nPts];

Show[
  ListPlot[
    {Transpose[{manningVals, continuityVals}]},
    PlotStyle → {Directive[RGBColor["#1565C0"], PointSize[0.018], Opacity[0.6]]},
    PlotLegends → {"Candidate sites"},
    AxesLabel → {"Manning Velocity (m/s)", "Continuity Velocity (m/s)"},
    PlotLabel → Style["Velocity Cross-Check", 14, Bold],
    PlotRange → {{0, 4}, {0, 4}},
    GridLines → Automatic,
    GridLinesStyle → Directive[GrayLevel[0.9]],
    ImageSize → 700
  ],
  Plot[x, {x, 0, 4},
    PlotStyle → {GrayLevel[0.6], Dashing[{0.02, 0.01]}]},
  Graphics[{RGBColor["#E53935"], Thickness[0.003],
    Line[{0, 0}, {4, 4}]}]
]

```

Out[85]=



```

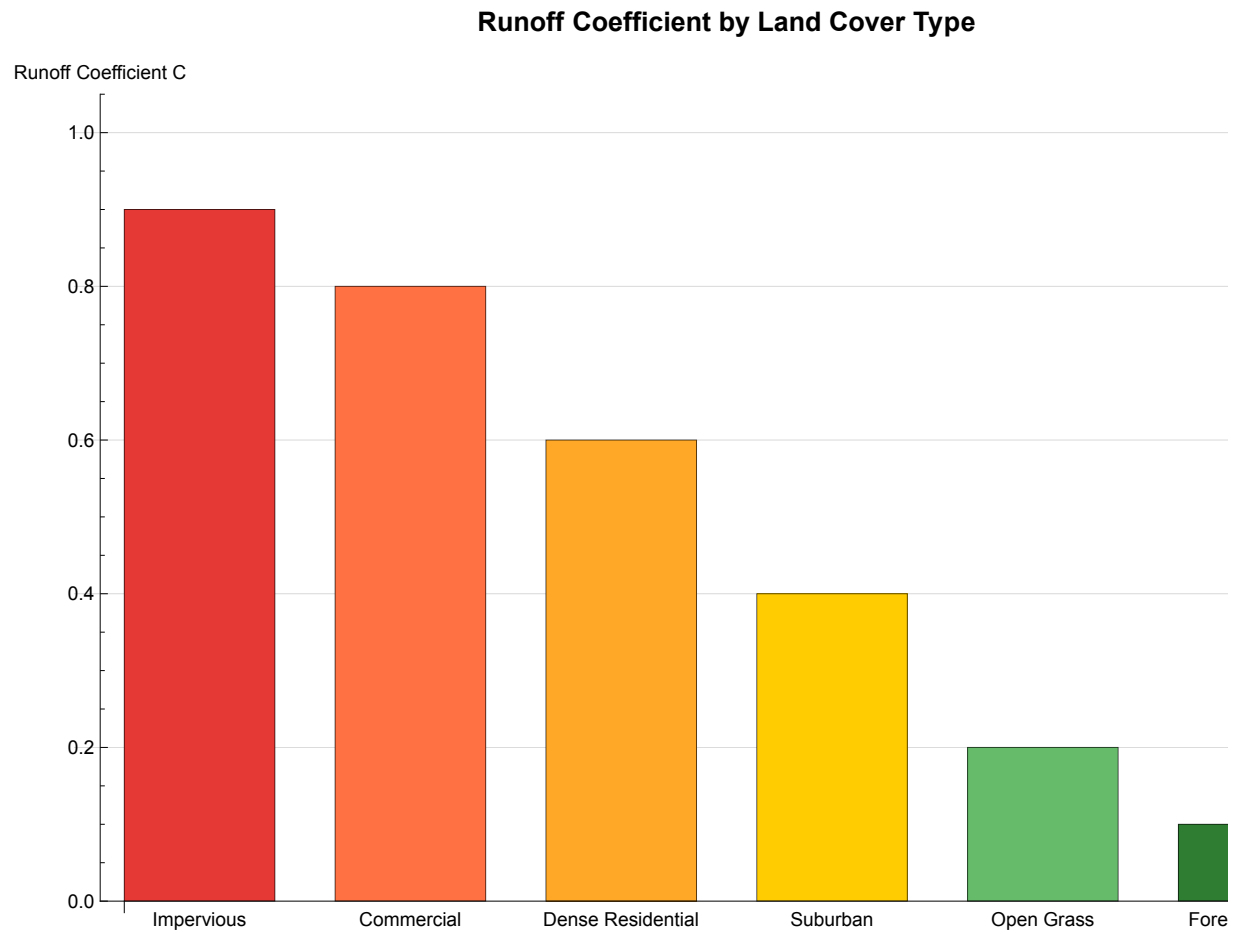
In[89]:= (* Runoff coefficient bar chart *)
data = {
  {"Impervious", 0.90},
  {"Commercial", 0.80},
  {"Dense Residential", 0.60},
  {"Suburban", 0.40},
  {"Open Grass", 0.20},
  {"Forest/Wetland", 0.10}
};

colors = {RGBColor["#E53935"], RGBColor["#FF7043"],
  RGBColor["#FFA726"], RGBColor["#FFCC02"],
  RGBColor["#66BB6A"], RGBColor["#2E7D32"]};

BarChart[
  data[[All, 2]],
  ChartLabels → Placed[Style[#, 10] & /@ data[[All, 1]], Below],
  ChartStyle → colors,
  BarSpacing → 0.4,
  PlotLabel → Style["Runoff Coefficient by Land Cover Type", 14, Bold],
  AxesLabel → {None, "Runoff Coefficient C"},
  PlotRange → {0, 1.05},
  GridLines → {None, {0.2, 0.4, 0.6, 0.8, 1.0}},
  GridLinesStyle → Directive[GrayLevel[0.85]],
  LabelStyle → Directive[10],
  ImagePadding → {{50, 20}, {60, 30}},
  ImageSize → 750
]

```

Out[91]=



```

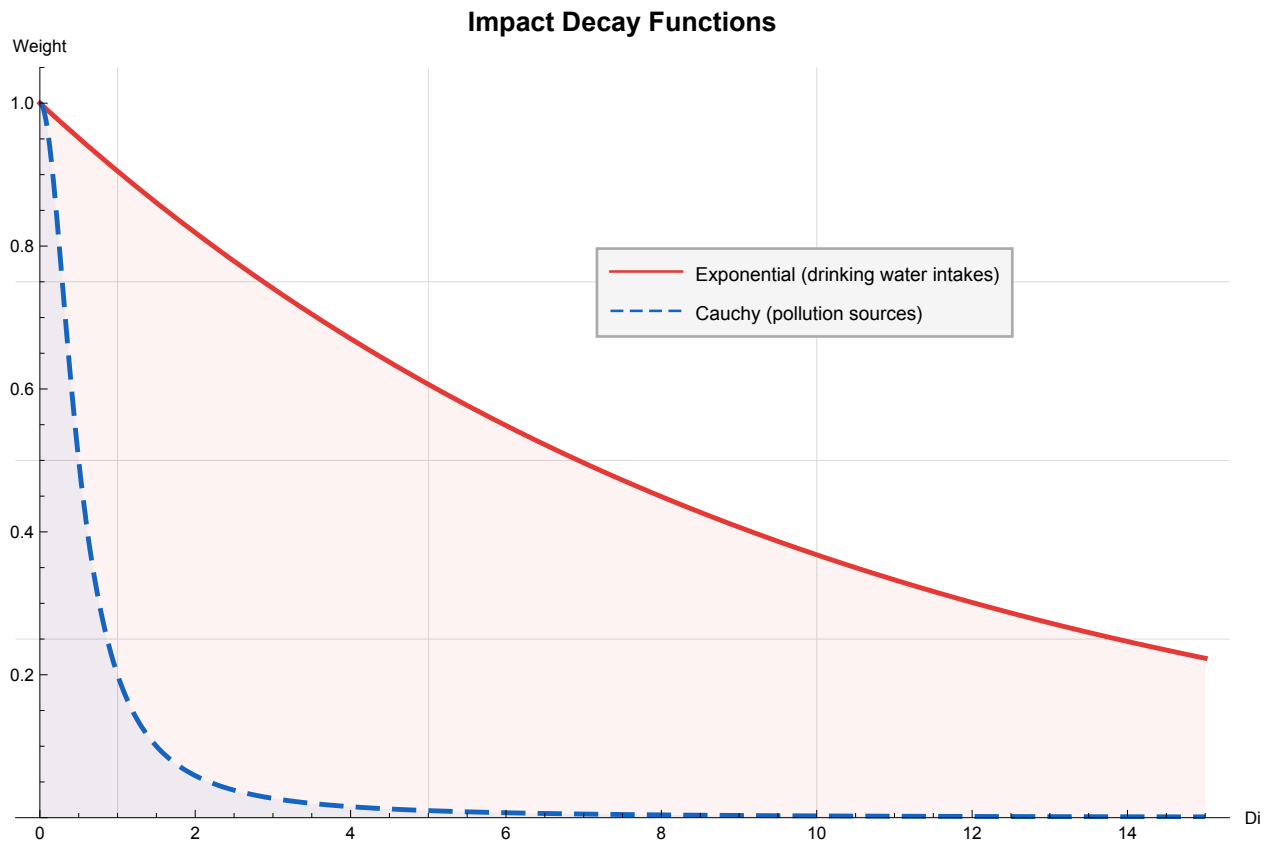
In[92]:= (* Exponential decay vs Cauchy decay *)
expDecay[d_] := Exp[-d/10];
cauchyDecay[d_] := 1/(1 + (d/0.5)^2);

Plot[{expDecay[d], cauchyDecay[d]},
  {d, 0, 15},
  PlotStyle → {
    {RGBColor["#E53935"], Thickness[0.004]},
    {RGBColor["#1565C0"], Thickness[0.004], Dashing[{0.02, 0.01]}}
  },
  PlotLegends → Placed[
    LineLegend[{
      "Exponential (drinking water intakes)",
      "Cauchy (pollution sources)"},
    LegendFunction → "Panel"],
    {0.65, 0.7}
  ],
  AxesLabel → {"Distance (km)", "Weight"},
  PlotLabel → Style["Impact Decay Functions", 14, Bold],
  Filling → {
    1 → {Axis, Directive[RGBColor["#E53935"], Opacity[0.06]]},
    2 → {Axis, Directive[RGBColor["#1565C0"], Opacity[0.06]]}
  },
  PlotRange → {0, 1.05},
  GridLines → {{1, 5, 10}, {0.25, 0.5, 0.75}},
  GridLinesStyle → Directive[GrayLevel[0.85]],
  ImageSize → 700
]

```



Out[94]=



```

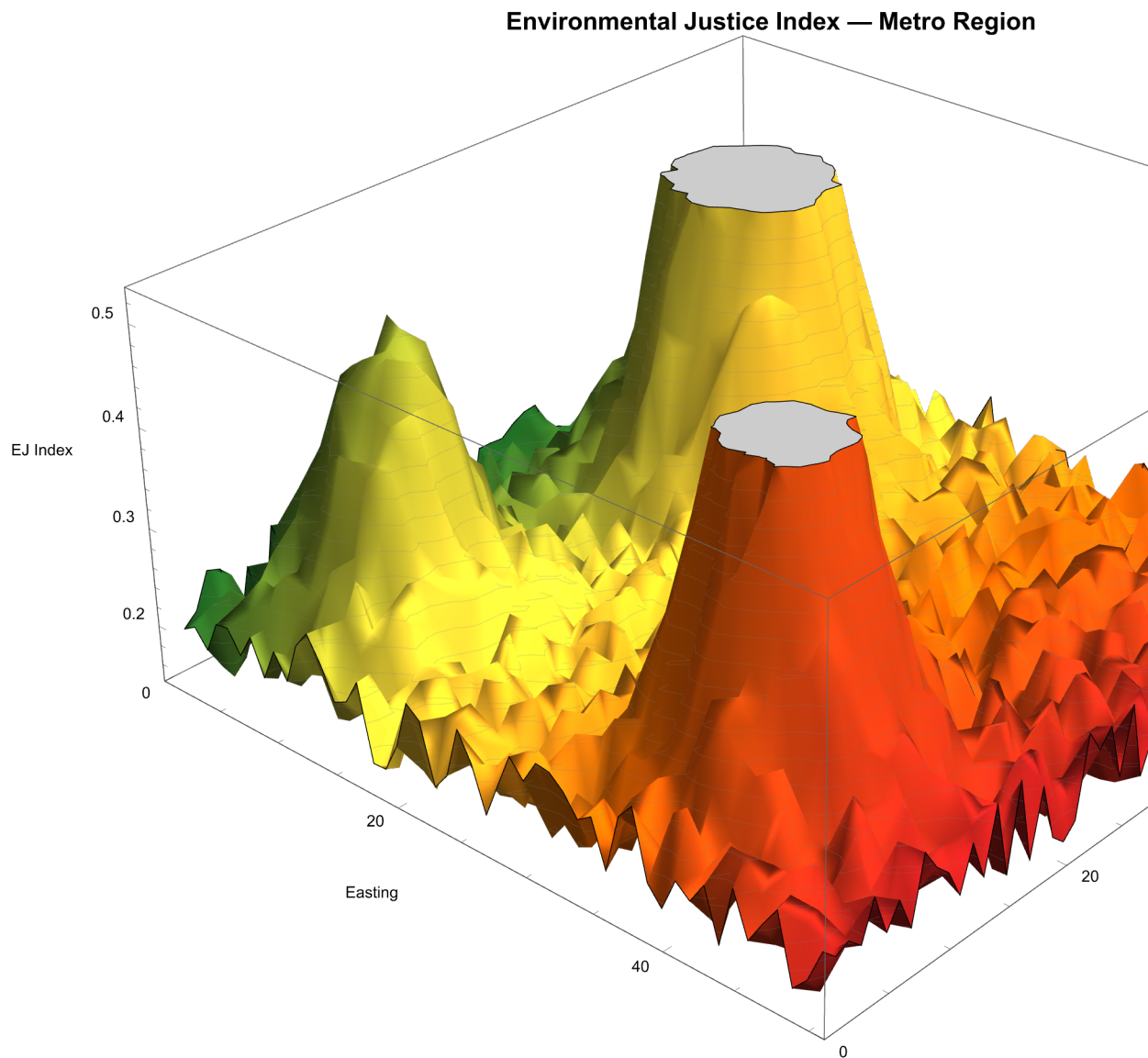
In[95]:= (* EJ Index surface across synthetic metro region *)
SeedRandom[33];
gridSize = 50;

ej = Table[
  0.2 +
  0.6 Exp[-((x - 15)^2 + (y - 35)^2)/40] +
  0.5 Exp[-((x - 38)^2 + (y - 12)^2)/30] +
  0.3 Exp[-((x - 10)^2 + (y - 10)^2)/25] +
  RandomReal[{-0.05, 0.05}],
  {y, 1, gridSize}, {x, 1, gridSize}
];

ListPlot3D[ej,
  ColorFunction -> (Blend[{
    RGBColor["#1B5E20"], RGBColor["#FDD835"],
    RGBColor["#FF6F00"], RGBColor["#B71C1C"]}, #] &),
  ColorFunctionScaling -> True,
  MeshFunctions -> {#3 &},
  MeshStyle -> Directive[GrayLevel[0.5], Opacity[0.2]],
  BoxRatios -> {1, 1, 0.5},
  PlotLabel -> Style["Environmental Justice Index - Metro Region", 14, Bold],
  AxesLabel -> {"Easting", "Northing", "EJ Index"},
  ViewPoint -> {2.2, -2.5, 1.8},
  Lighting -> "Neutral",
  ImageSize -> 800
]

```

Out[98]=



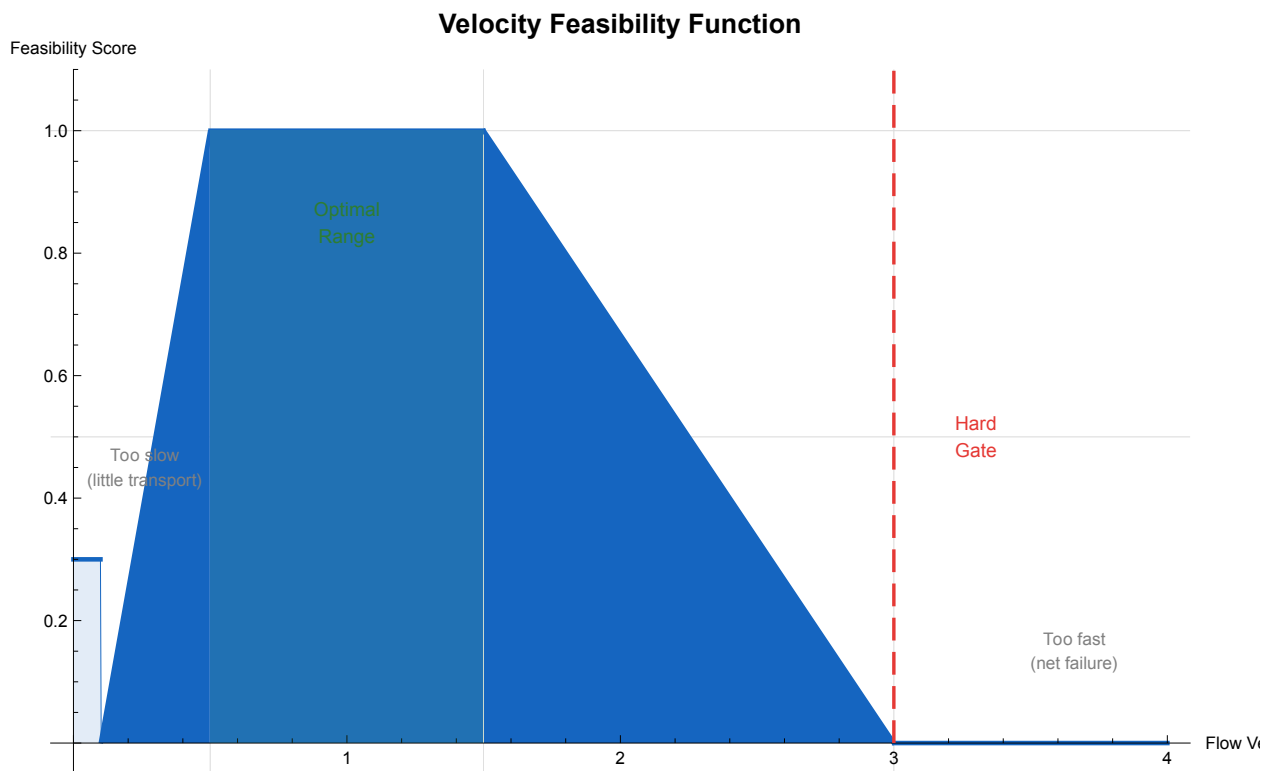
```

In[99]:= (* Piecewise velocity feasibility *)
feasibility[v_] := Piecewise[{
  {0.3, v < 0.1},
  {(v - 0.1)/0.4, 0.1 ≤ v < 0.5},
  {1.0, 0.5 ≤ v ≤ 1.5},
  {(3.0 - v)/1.5, 1.5 < v ≤ 3.0},
  {0, v > 3.0}
}];

Plot[feasibility[v], {v, 0, 4},
  PlotStyle → {RGBColor["#1565C0"], Thickness[0.004]},
  Filling → Axis,
  FillingStyle → Directive[RGBColor["#1565C0"], Opacity[0.12]],
  AxesLabel → {"Flow Velocity (m/s)", "Feasibility Score"},
  PlotLabel → Style["Velocity Feasibility Function", 14, Bold],
  PlotRange → {-0.05, 1.1},
  Epilog → {
    {RGBColor["#66BB6A"], Opacity[0.15],
      Rectangle[{0.5, 0}, {1.5, 1.0}]},
    {RGBColor["#E53935"], Dashing[{0.02, 0.01}], Thickness[0.003],
      Line[{3.0, 0}, {3.0, 1.1}]},
    Text[Style["Optimal\nRange", 10, RGBColor["#2E7D32"]], {1.0, 0.85}],
    Text[Style["Hard\nGate", 10, RGBColor["#E53935"]], {3.3, 0.5}],
    Text[Style["Too slow\n(little transport)", 9, GrayLevel[0.5]], {0.05, 0.45}, {-1, 0}],
    Text[Style["Too fast\n(net failure)", 9, GrayLevel[0.5]], {3.5, 0.15}, {-1, 0}]
  },
  GridLines → {{0.5, 1.5, 3.0}, {0.5, 1.0}},
  GridLinesStyle → Directive[GrayLevel[0.85]],
  ImageSize → 700
]

```

Out[100]=



In[101]:=

```

(* Feasibility surface: width x velocity *)
widthScore[w_] := Piecewise[{
  {0.2, w < 1},
  {1.0, 1 ≤ w ≤ 15},
  {Max[0, (50 - w)/35], 15 < w ≤ 50},
  {0, w > 50}
}];

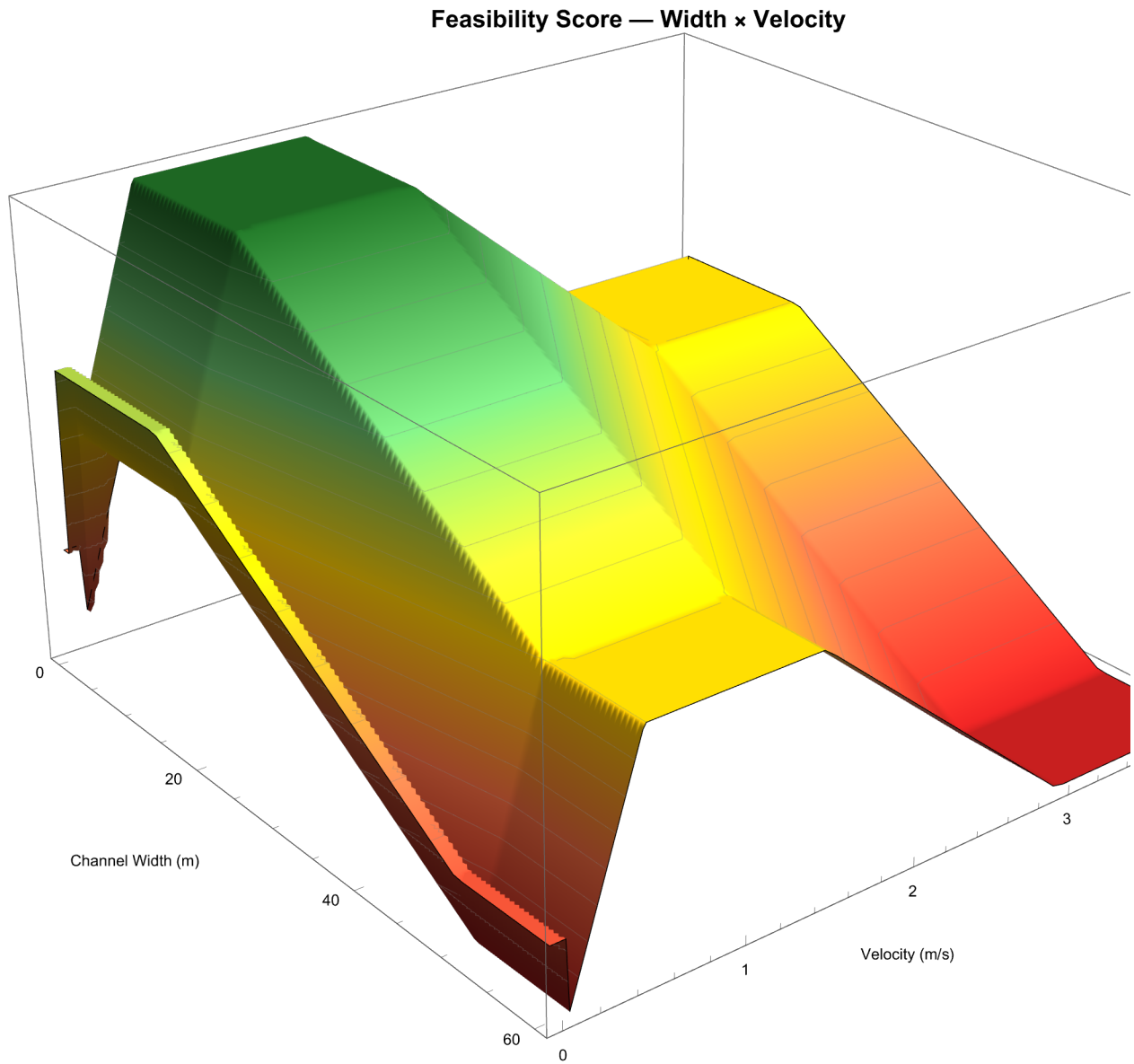
velFeas[v_] := Piecewise[{
  {0.3, v < 0.1},
  {(v - 0.1)/0.4, 0.1 ≤ v < 0.5},
  {1.0, 0.5 ≤ v ≤ 1.5},
  {(3.0 - v)/1.5, 1.5 < v ≤ 3.0},
  {0, v > 3.0}
}];

feasComposite[w_, v_] := 0.5 widthScore[w] + 0.5 velFeas[v];

Plot3D[feasComposite[w, v],
  {w, 0, 60}, {v, 0, 4},
  ColorFunction → (Blend[{
    RGBColor["#B71C1C"], RGBColor["#FF7043"],
    RGBColor["#FFCC02"], RGBColor["#66BB6A"],
    RGBColor["#1B5E20"]}], #3] &),
  ColorFunctionScaling → True,
  MeshFunctions → {#3 &},
  MeshStyle → Directive[GrayLevel[0.5], Opacity[0.3]],
  AxesLabel → {"Channel Width (m)", "Velocity (m/s)", "Feasibility"},
  PlotLabel → Style["Feasibility Score – Width × Velocity", 14, Bold],
  BoxRatios → {1, 1, 0.6},
  ViewPoint → {2.5, -2, 1.5},
  Lighting → "Neutral",
  PlotRange → {0, 1},
  PlotPoints → 60,
  ImageSize → 800,
  Exclusions → None
]

```

Out[104]=



In[105]:=

```

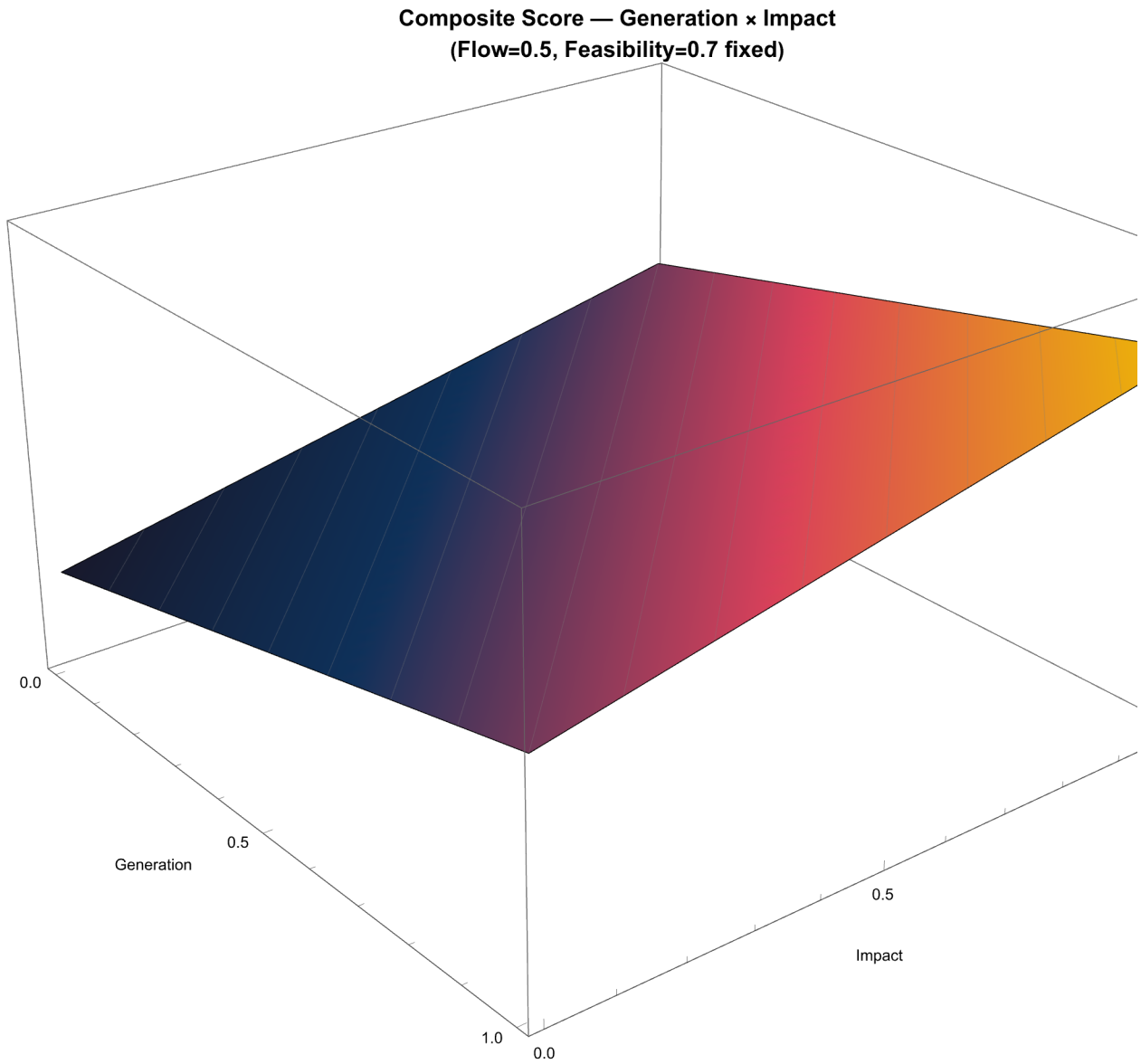
(* Composite score surface: Generation x Impact *)
composite[gen_, imp_] :=
  100 (0.30 gen + 0.25 * 0.5 + 0.30 imp + 0.15 * 0.7);

Plot3D[composite[gen, imp],
  {gen, 0, 1}, {imp, 0, 1},
  ColorFunction -> (Blend[{
    RGBColor["#1a1a2e"], RGBColor["#0f3460"],
    RGBColor["#e94560"], RGBColor["#ffcc00"]}, #3] &),
  ColorFunctionScaling -> True,
  MeshFunctions -> {#3 &},
  MeshStyle -> Directive[GrayLevel[0.5], Opacity[0.3]],
  AxesLabel -> {"Generation", "Impact", "Composite Score"},
  PlotLabel -> Style[
    "Composite Score - Generation x Impact\n(Flow=0.5, Feasibility=0.7 fixed)", 13, Bold]
  BoxRatios -> {1, 1, 0.6},
  ViewPoint -> {2.5, -2, 1.5},
  Lighting -> "Neutral",
  PlotRange -> {0, 100},
  PlotPoints -> 40,
  ImageSize -> 800
]

```



Out[106]=



In[124]:=

```

(* Hard gate constraint filtering *)
SeedRandom[55];
nCandidates = 200;

widths = RandomReal[{1, 80}, nCandidates];
velocities = RandomReal[{0.05, 5}, nCandidates];
ownership = RandomChoice[{"public", "public", "public", "private"}, nCandidates];

passGate = MapThread[
  (#1 ≤ 50 && #2 ≤ 3.0 && #3 == "public") &,
  {widths, velocities, ownership}
];

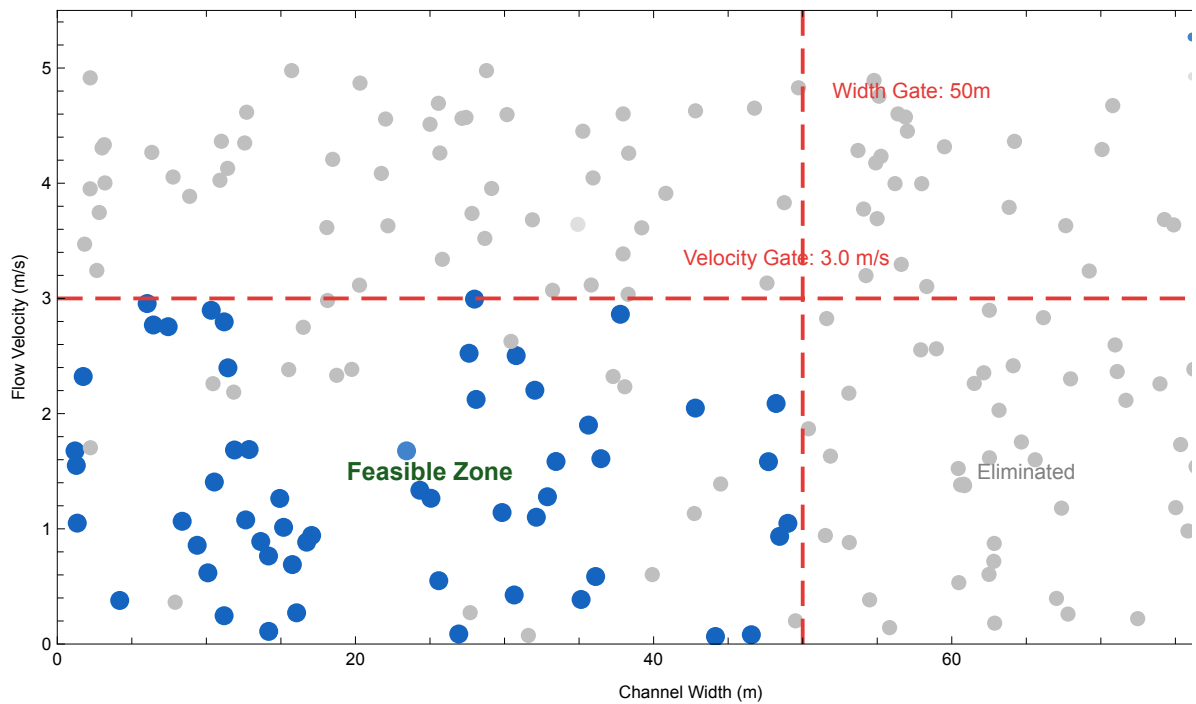
feasiblePts = Pick[Transpose[{widths, velocities}], passGate];
eliminatedPts = Pick[Transpose[{widths, velocities}], passGate, False];

Show[
  ListPlot[{feasiblePts, eliminatedPts},
    PlotStyle → {
      Directive[RGBColor["#1565C0"], PointSize[0.015], Opacity[0.8]],
      Directive[GrayLevel[0.75], PointSize[0.012], Opacity[0.5]]
    },
    PlotRange → {{0, 85}, {0, 5.5}},
    AspectRatio → 0.5,
    Frame → True,
    FrameLabel → {"Channel Width (m)", "Flow Velocity (m/s)"},
    PlotLabel → Style["Hard Gate Constraint Filtering", 14, Bold],
    ImageSize → 750,
    ImagePadding → {{60, 30}, {50, 40}},
    PlotLegends → Placed[{"Feasible", "Eliminated"}, {Right, Top}],
    Background → White
  ],
  Graphics[{
    {RGBColor["#E53935"], Thickness[0.003], Dashing[{0.02, 0.01]},
      Line[{{50, 0}, {50, 5.5}}]},
    {RGBColor["#E53935"], Thickness[0.003], Dashing[{0.02, 0.01]},
      Line[{{0, 3.0}, {85, 3.0}}]},
    Text[Style["Width Gate: 50m", 11, RGBColor["#E53935"]], {52, 4.8}, {-1, 0}],
    Text[Style["Velocity Gate: 3.0 m/s", 11, RGBColor["#E53935"]], {42, 3.25}, {-1, -1}],
    Text[Style["Feasible Zone", 13, Bold, RGBColor["#1B5E20"]], {25, 1.5}],
    Text[Style["Eliminated", 11, GrayLevel[0.5]], {65, 1.5}]
  ]
]

```

Out[132]=

## Hard Gate Constraint Filtering



In[167]:=

```
(* Dirichlet samples projected onto ternary diagram *)
SeedRandom[42];

baseline = {0.30, 0.30, 0.15};
kappa = 10;
alpha = kappa * baseline;

(* Wolfram DirichletDistribution[{a1,a2,a3}] returns 2-component vectors;
   the third component is 1 - x1 - x2. Reconstruct full 3-vectors. *)
raw = RandomVariate[DirichletDistribution[alpha], 500];
samples = Append[#, 1 - Total[#]] & /@ raw;

(* Ternary projection: maps {a, b, c} onto 2D equilateral triangle *)
ternary[{a_, b_, c_}] := With[{s = a + b + c},
  {(2 b + c) / (2 s), c Sqrt[3] / (2 s)}
];

pts = ternary /@ samples;
baselinePt = ternary[baseline];

(* Triangle corners *)
```

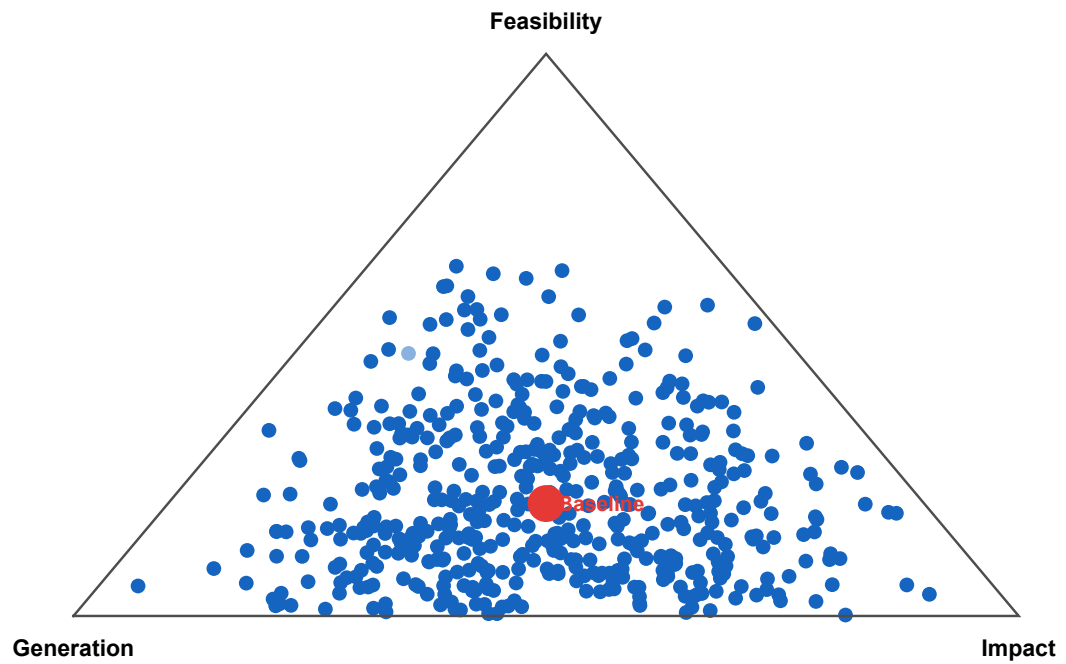
```

cA = ternary[{1, 0, 0}];
cB = ternary[{0, 1, 0}];
cC = ternary[{0, 0, 1}];

Show[
  ListPlot[pts,
    PlotStyle → Directive[RGBColor["#1565C0"], PointSize[0.012], Opacity[0.5]],
    PlotRange → {{-0.15, 1.15}, {-0.12, 1.05}},
    Axes → False,
    ImageSize → 700,
    ImagePadding → {{30, 30}, {30, 40}},
    Background → White,
    PlotLabel → Style["Dirichlet Weight Perturbations ( $\kappa = 10$ )", 14, Bold]
  ],
  Graphics[{
    (* Triangle outline *)
    {GrayLevel[0.3], Thickness[0.002], Line[{cA, cB, cC, cA}]},
    (* Baseline marker – on top of blue cloud *)
    {RGBColor["#E53935"], PointSize[0.03], Point[baselinePt]},
    {RGBColor["#E53935"],
      Text[Style[" Baseline", 11, Bold], baselinePt, {-1, 0}]},
    (* Vertex labels *)
    Text[Style["Generation", 12, Bold], cA + {0, -0.05}],
    Text[Style["Impact", 12, Bold], cB + {0, -0.05}],
    Text[Style["Feasibility", 12, Bold], cC + {0, 0.05}]
  ]
]

```

Out[179]=

**Dirichlet Weight Perturbations ( $\kappa = 10$ )**

In[183]:=

```

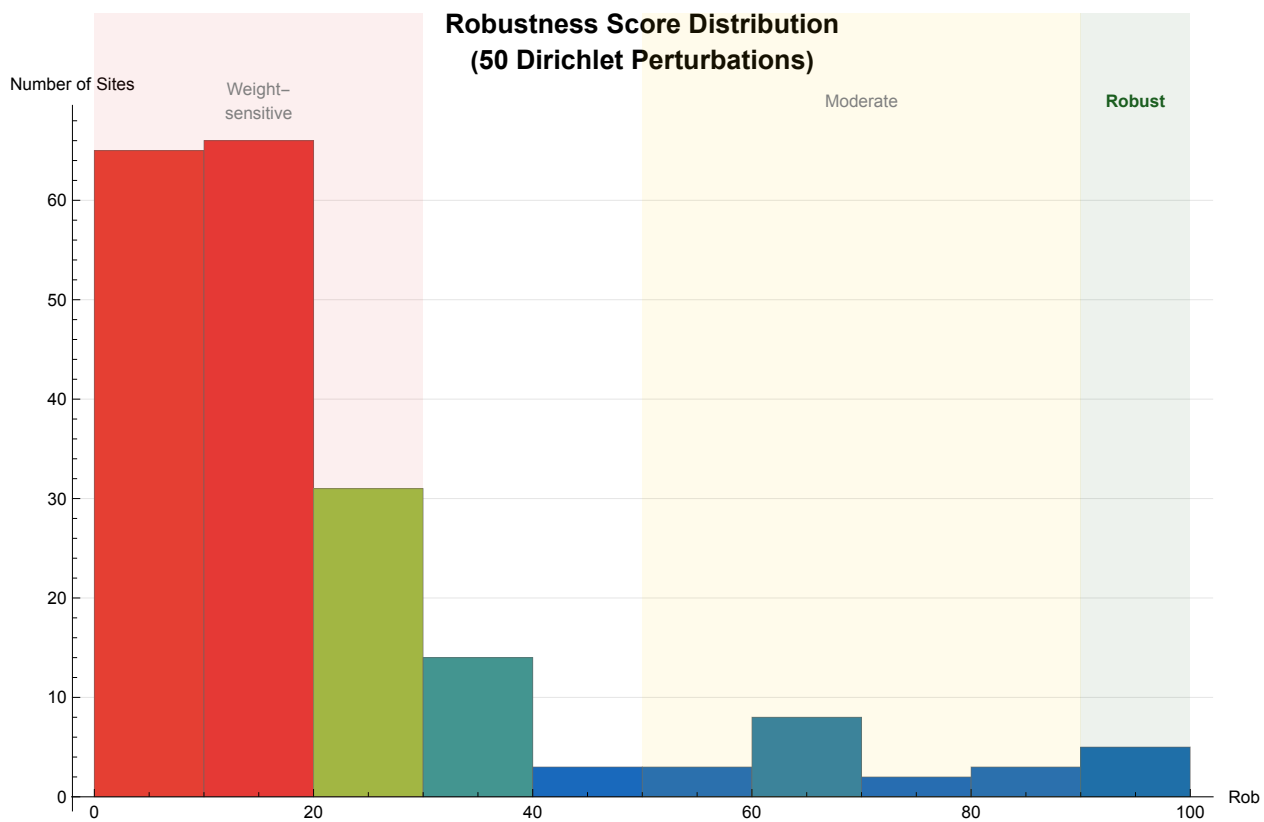
(* Robustness score distribution *)
SeedRandom[77];

robustness = Join[
  RandomReal[BetaDistribution[1.5, 8], 180],
  RandomReal[BetaDistribution[3, 2], 15],
  RandomReal[BetaDistribution[12, 1.5], 5]
] * 100;

Histogram[robustness,
  {10},
  ColorFunction -> (Blend[{RGBColor["#1565C0"], RGBColor["#66BB6A"],
    RGBColor["#FFCC02"], RGBColor["#E53935"]}], #] &),
  AxesLabel -> {"Robustness (%)", "Number of Sites"},
  PlotLabel -> Style[Column[{"Robustness Score Distribution", "(50 Dirichlet Perturbations"}],
  ChartBaseStyle -> EdgeForm[GrayLevel[0.4]],
  GridLines -> {None, Automatic},
  GridLinesStyle -> Directive[GrayLevel[0.9]],
  Epilog -> {
    {RGBColor["#E53935"], Opacity[0.08], Rectangle[{0, 0}, {30, 80}]},
    {RGBColor["#FFCC02"], Opacity[0.08], Rectangle[{50, 0}, {90, 80}]},
    {RGBColor["#1B5E20"], Opacity[0.08], Rectangle[{90, 0}, {100, 80}]},
    Text[Style["Weight-\nsensitive", 9, GrayLevel[0.5]], {15, 70}],
    Text[Style["Moderate", 9, GrayLevel[0.5]], {70, 70}],
    Text[Style["Robust", 9, Bold, RGBColor["#1B5E20"]], {95, 70}]
  },
  ImageSize -> 700
]

```

Out[185]=



In[186]:=

```

(* Rank stability heatmap *)
SeedRandom[88];

nSites = 20;
nPerturbations = 50;

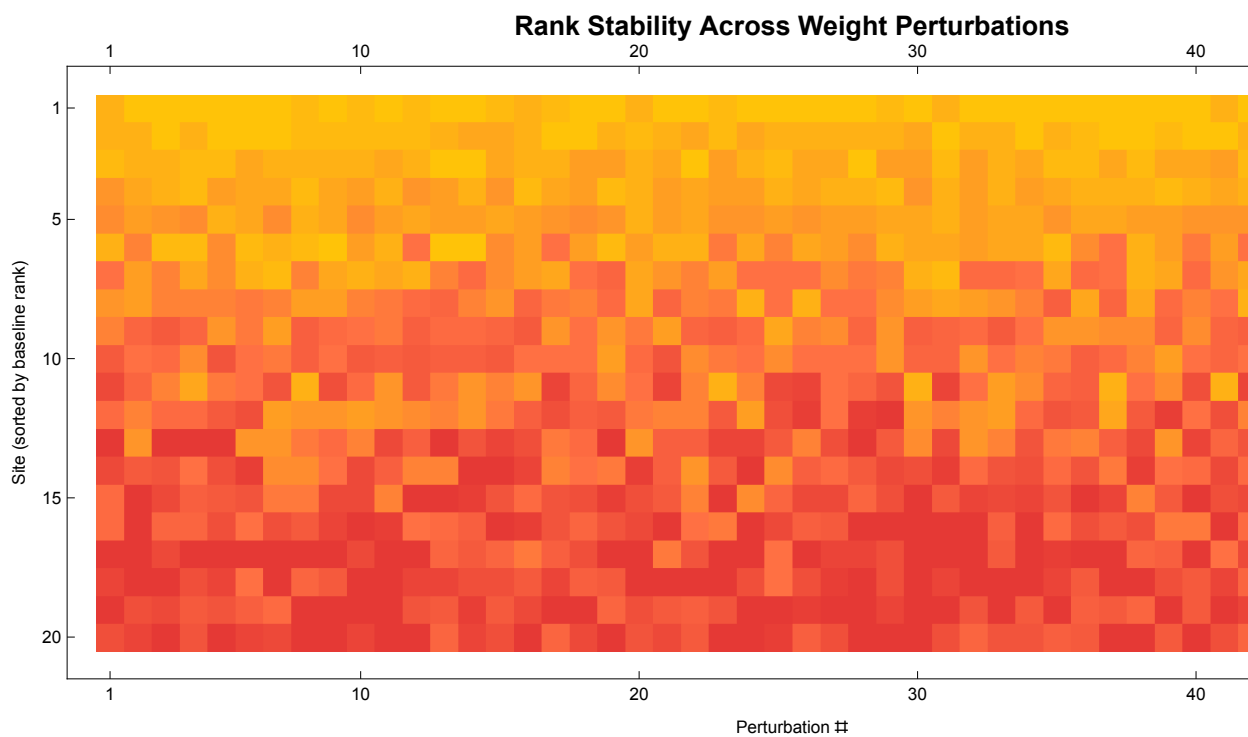
ranks = Table[
  Module[{base = i, noise = If[i ≤ 5, 2, If[i ≤ 10, 5, 8]]},
    Clip[Round[base + RandomReal[{-noise, noise}]], {1, nSites}]
  ],
  {i, nSites}, {j, nPerturbations}
];

MatrixPlot[ranks,
  ColorFunction → (Blend[{
    RGBColor["#1B5E20"], RGBColor["#66BB6A"],
    RGBColor["#FFCC02"], RGBColor["#FF7043"],
    RGBColor["#E53935"]}, #] &),
  ColorFunctionScaling → True,
  FrameLabel → {"Site (sorted by baseline rank)", "Perturbation #"},
  PlotLabel → Style["Rank Stability Across Weight Perturbations", 14, Bold],
  PlotLegends → BarLegend[{
    Blend[{RGBColor["#1B5E20"], RGBColor["#66BB6A"],
      RGBColor["#FFCC02"], RGBColor["#FF7043"],
      RGBColor["#E53935"]}, #] &, {1, nSites}},
    LegendLabel → "Rank"],
  ImageSize → 800
]

```



Out[190]=



In[191]:=

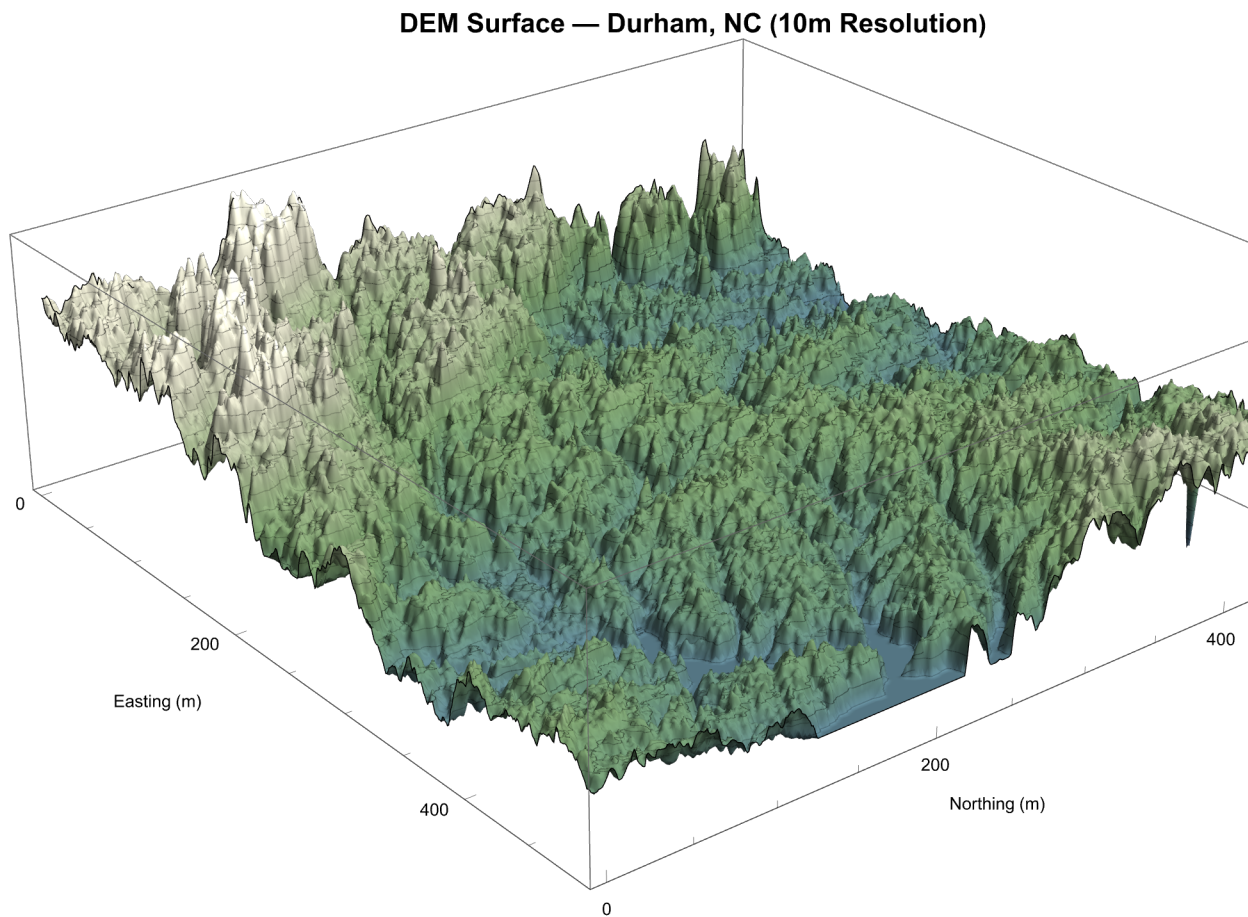
```

(* 3D terrain surface – Durham, NC *)
demDurham = QuantityMagnitude[GeoElevationData[
  Entity["City", {"Durham", "NorthCarolina", "UnitedStates"}],
  GeoRange → Quantity[15, "Kilometers"],
  GeoProjection → Automatic
]];

ListPlot3D[demDurham,
  PlotRange → All,
  ColorFunction → "AlpineColors",
  MeshFunctions → {#3 &},
  MeshStyle → Opacity[0.2],
  Lighting → "Neutral",
  BoxRatios → {1, 1, 0.3},
  PlotLabel → Style["DEM Surface – Durham, NC (10m Resolution)", 14, Bold],
  AxesLabel → {"Easting (m)", "Northing (m)", "Elevation (m)"},
  ImageSize → 800,
  ViewPoint → {2.5, -2, 1.5}
]

```

Out[192]=



In[193]:=

```

(* Dashboard pipeline as a flowchart *)
nodes = {"City Selection", "Overpass API", "Waterway Geometry",
  "Candidate Generation", "Hard Gate Filter",
  "Scoring (28 params)", "Risk Filter",
  "Map Rendering"};

edges = Thread[nodes[;; -2] → nodes[2 ;;]];

colors = {RGBColor["#58B09C"], RGBColor["#42A5F5"], RGBColor["#42A5F5"],
  RGBColor["#66BB6A"], RGBColor["#E53935"], RGBColor["#FF7043"],
  RGBColor["#FFCC02"], RGBColor["#58B09C"]};

Graph[edges,
  VertexLabels → Placed["Name", Center],
  VertexLabelStyle → Directive[9, Bold, White],
  VertexStyle → Thread[nodes → (Directive[#, EdgeForm[None]] & /@ colors)],
  VertexSize → 0.75,
  VertexShapeFunction → "RoundedRectangle",
  EdgeStyle → Directive[GrayLevel[0.5], Thickness[0.004], Arrowheads[0.04]],
  GraphLayout → {"LayeredDigraphEmbedding", "Orientation" → Left},
  PlotLabel → Style["Dashboard Client-Side Scoring Pipeline", 14, Bold],
  ImageSize → 900
]

```

Out[196]=

### Dashboard Client-Side Scoring Pipeline

